

自动驾驶和避障

一、实验内容

利用自动驾驶数据训练自动驾驶模型，实现车辆沿着轨道行驶和避障功能。

二、实验目标

- 掌握自动驾驶和避障的实现的流程
- 利用卷积神经网络（CNN）训练自动驾驶和避障模型
- 锻炼实战思维，提高实战技能

三、实验环境

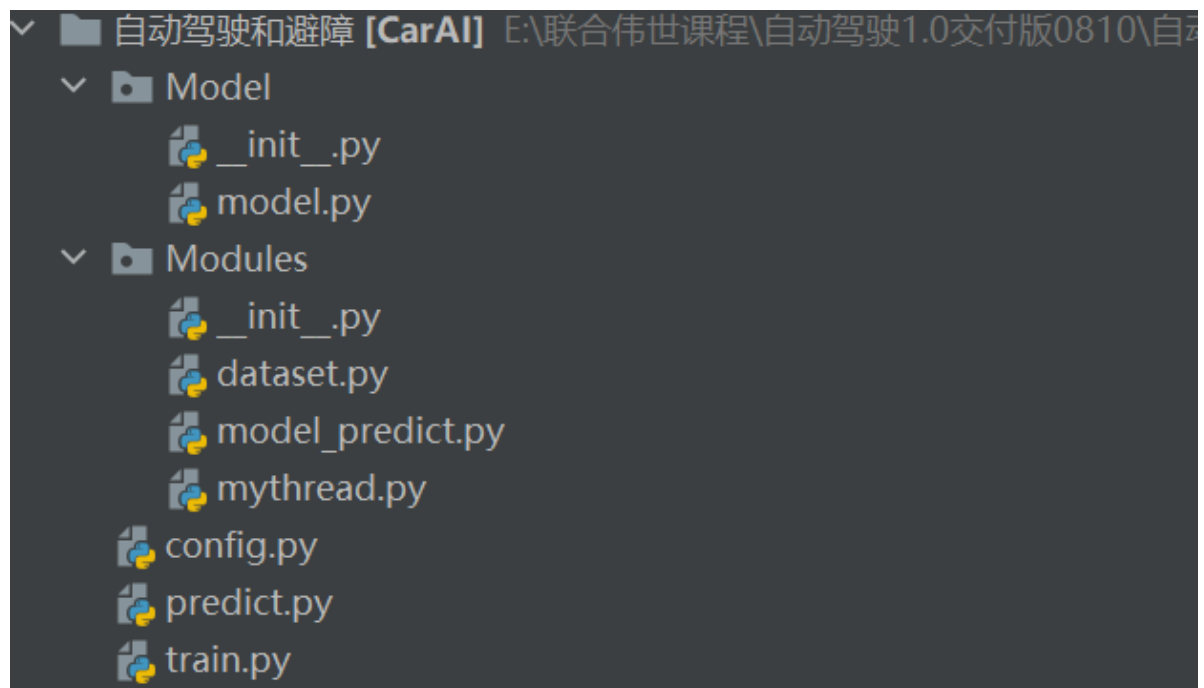
- 操作系统：Windows10
- 工具软件：PyCharm 、Python 3.6.13
- 核心库：

```
Keras==2.2.4  
tensorflow==1.8.0
```

详细可参考项目中的依赖库描述文件：requirements.txt 文件

四、实验原理

1、项目结构介绍



- model.py : 创建模型文件
- dataset.py: 数据处理文件
- model_predict.py: 训练模型代码
- thread.py: 线程文件
- predict.py: 预测代码
- train.py: 训练代码

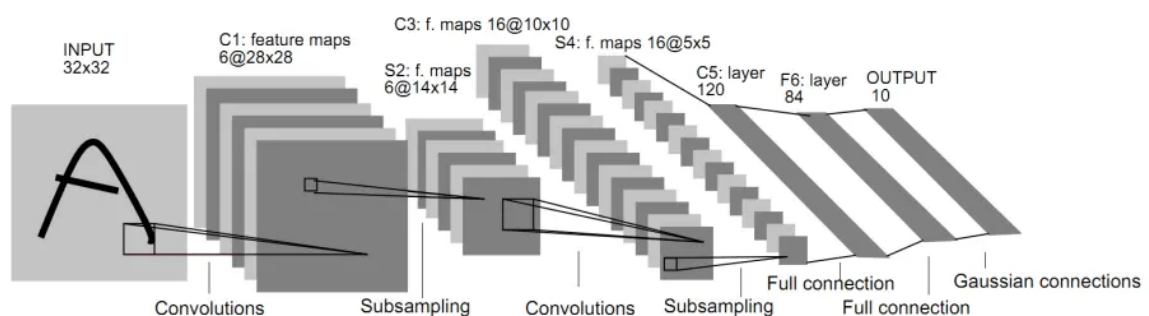
2、自动驾驶和避障实现步骤

1. 数据采集（采集行驶中的图片、角度、速度），项目中我们使用图片和角度两个特征值。
2. 模型训练（通过图像、角度两个特征值利用CNN网络训练模型）
3. 模型推理（输入特征为图片，经过模型推理得到输出特征值角度）

3、CNN介绍

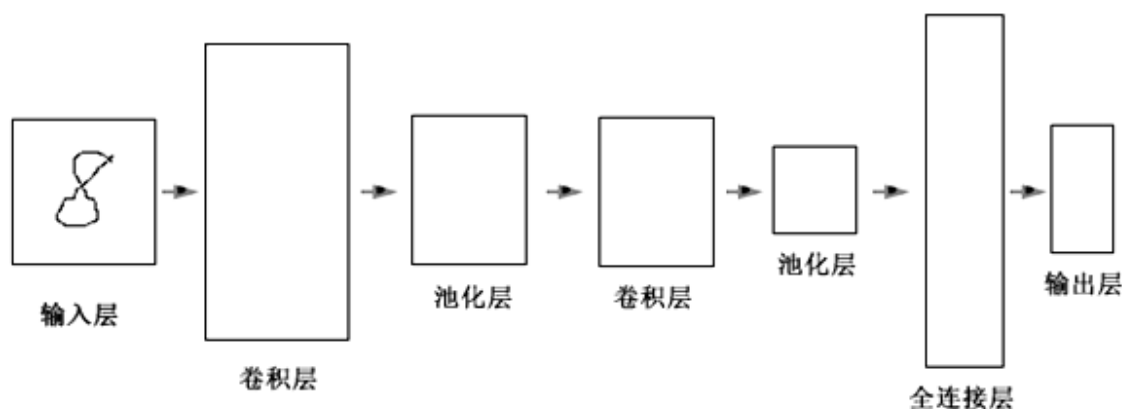
卷积神经网络（Convolutional Neural NetWork,简称 CNN）是一种包含卷积计算且具有深度结构的前馈神经网络，被广泛应用在图像识别、自然语言处理和语音识别等领域。本实验中自动驾驶轨道和障碍物的识别就使用的是CNN来实现。

神经网络结构



CNN网络核心的结构层如下，实际应用中各层的数量可以灵活配置

- 输入层
- 卷积层
- 池化层
- 全连接FC层
- 输出层



本实验构建了8层网络

- 5层卷积
- 4层全连接

```
# RaKnet 神经网络
def get_classify_model(self, num_class):
    input_img = Input(shape=(conf.IMG_DEPTH, conf.IMG_HEIGHT, conf.IMG_WIDTH), name='input_img')
    if K.image_data_format() == "channels_last": # 确认输入维度
        input_img = Input(shape=(conf.IMG_HEIGHT, conf.IMG_WIDTH, conf.IMG_DEPTH), name='input_img')
    x = input_img

    x = Conv2D(20, (5, 5), strides=(2, 2), padding='same', activation='relu')(x)
    x = Conv2D(50, (3, 3), strides=(2, 2), padding='same', activation='relu')(x)
    x = Conv2D(50, (3, 3), strides=(1, 1), padding='same', activation='relu')(x)
    x = Conv2D(80, (3, 3), strides=(2, 2), padding='same', activation='relu')(x)
    x = Conv2D(80, (3, 3), strides=(1, 1), padding='same', activation='relu')(x)

    x = Flatten(name='flattened')(x)
    x = Dense(100, activation='relu')(x)
    x = Dropout(.1)(x)
    x = Dense(50, activation='relu')(x)
    x = Dropout(.1)(x)

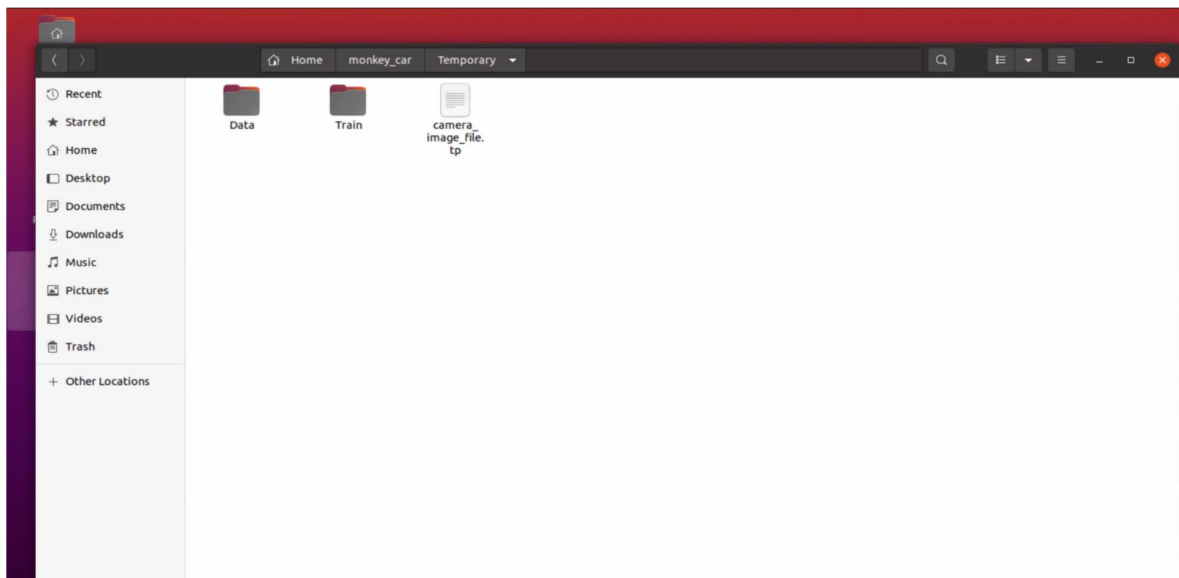
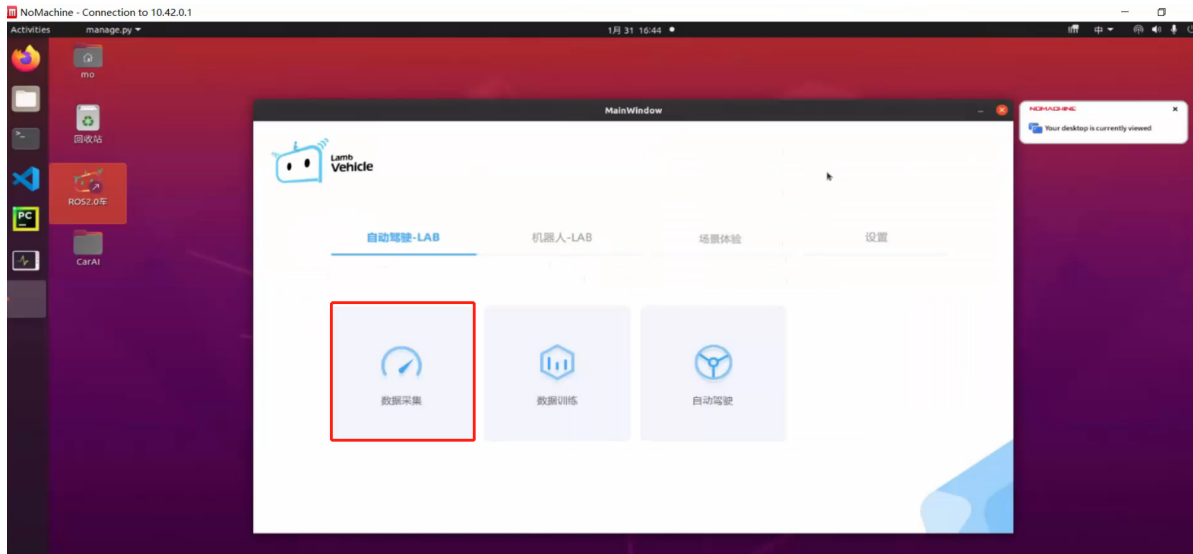
    angle = Dense(num_class, activation='softmax', name='angle')(x)
    throttle = Dense(1, activation='relu', name='throttle')(x)
    model = Model(inputs=[input_img], outputs=[angle, throttle])

    return model
```

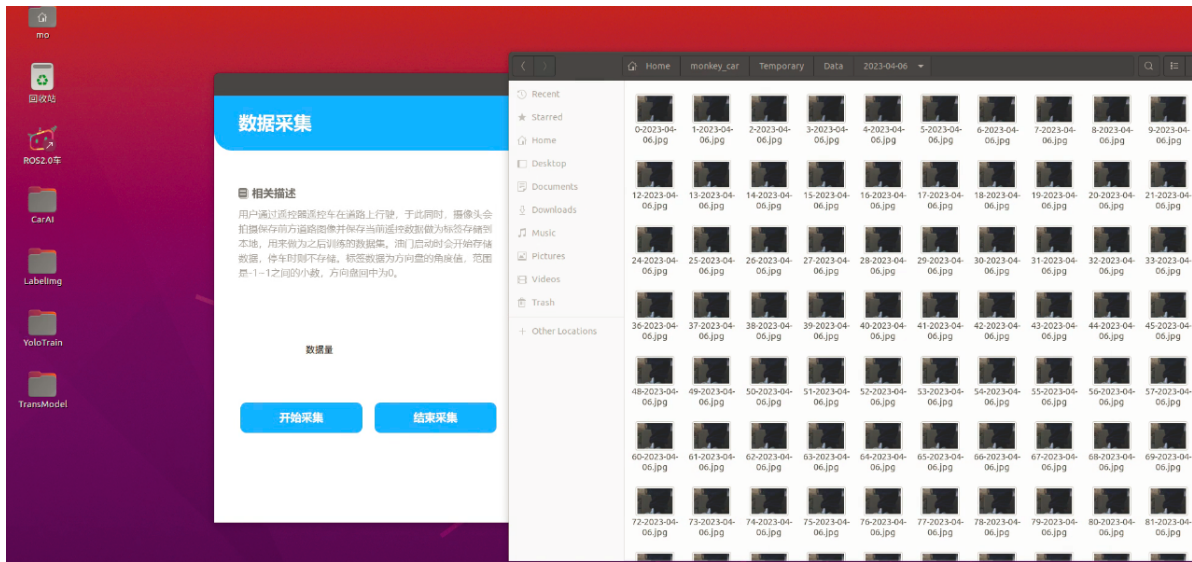
图像经过层层卷积，核心特征被提取出来，在全连接层实现分类任务，将良好的行车路线角度计算出来，最终实现沿着训练路线自动行驶。

4、数据集介绍

标签数据为方向盘的角度值，范围是-1~1 之间的小数，方向盘回中为 0。采集的数据默认存到了项目的 Data 文件夹里，绝对路径也就是 /Home/monkey_car/Temporary/Data，进入此文件夹会看到你采集的图片与标签，其记录的是对应前方道路图片应该的转向角度信息。



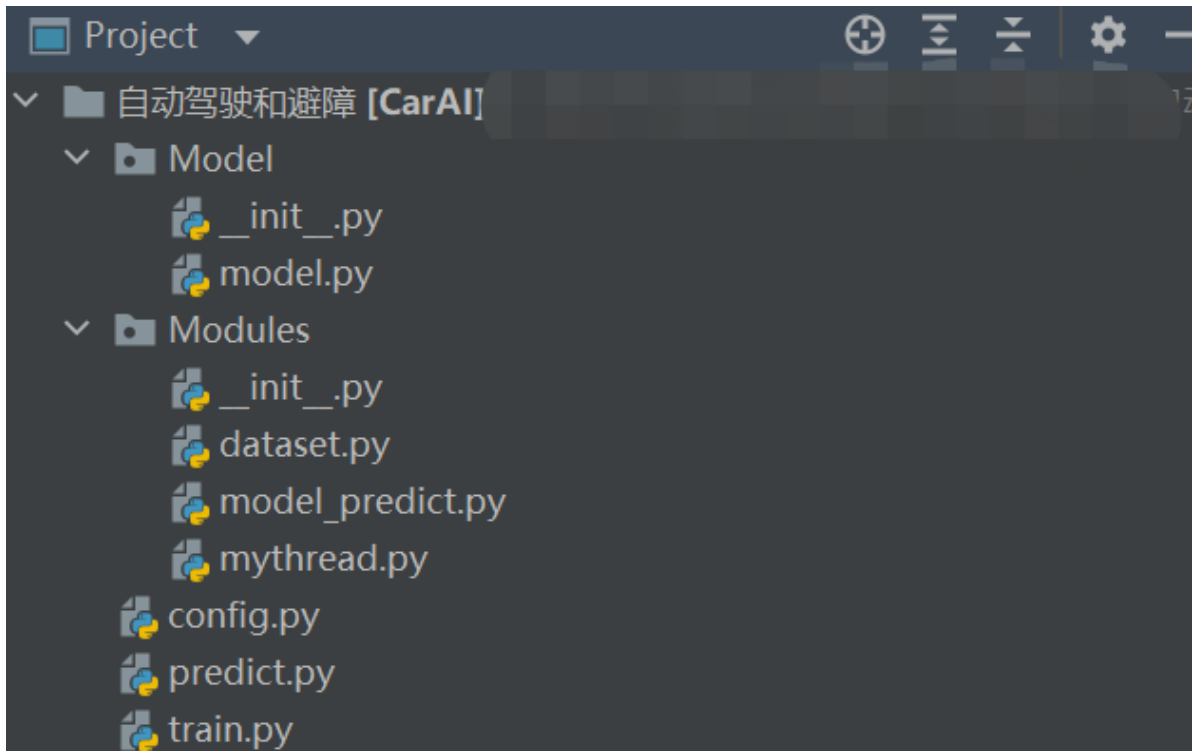
进入此文件夹会看到你采集的图片与标签，其记录的是对应前方道路图片应该的转向角度信息，这样的数据集之后会送入一个神经网络中训练生成最终的模型。



五、实验步骤

打开PyCharm开发环境，新建项目，命名为自动行驶和避障。

步骤1、构建项目结构




```

        error_msg = '{} optimizer is not support, please check you input
optimizer types!'
        print('[ 20004 ] - {}'.format(error_msg))
        raise Exception(error_msg)

# 训练类型
print('[ 20005 ] - Model Compile...')

model.compile(optimizer=opt, metrics=["acc"],
               loss={'angle': 'categorical_crossentropy',
                     'throttle': 'mean_absolute_error'},
               loss_weights={'angle': 0.9, 'throttle': .01}
               )

# 展示网络结构
print('[ 20006 ] - show Model Summary...')
RaknetModel().show_model_summary(model)

# 打印参数
print('[ 20007 ] - Print Training Parameters...')

# 设置early_stop
if early_stop:
    # patience = int(epochs * 0.1) if epochs > 100 else 10
    print('[ 20009 ] - Early Stop, Patience : {}'.format(patience))
    patience_type = 'loss' # val_loss
    callbacks_list =
[keras.callbacks.EarlyStopping(monitor=patience_type, patience=patience,
verbose=0),
                           keras.callbacks.ModelCheckpoint(save_model_path,
monitor=patience_type, verbose=0,

save_best_only=True, mode='min')]
    print('[ 20010 ] - Callbacks List : {}'.format(callbacks_list))
else:
    callbacks_list = [keras.callbacks.ModelCheckpoint(save_model_path,
monitor='val_loss', verbose=0,

save_best_only=True, mode='min')]
    print('[ 20011 ] - Callbacks List : {}'.format(callbacks_list))

# 开始训练
multipro = False if num_workers == 1 else True
print('[ 20012 ] - Multi Process Train : {}'.format(multipro))
print('[ 20013 ] - Training...')

model.fit(x=train_x,
          y=[train_angle_y, train_throttle_y],
          epochs=epochs,
          batch_size=batch_size,
          verbose=2,
          validation_data=(test_x, [test_angle_y, test_throttle_y]),
          callbacks=callbacks_list,
          workers=num_workers,
          use_multiprocessing=multipro,
          )

def go(self,

```

```

        save_model_path,
        train_folder,
        batch_size=32,
        loads='False',
        epochs=32,
        lr=1e-3,
        early_stop='True',
        num_workers=1,
        optimizer='adam',
        gpu_num='CPU',
        patience=15):

    print('[ 20014 ] - Save Model Path : {}'.format(save_model_path))
    print('[ 20015 ] - Train Folder : {}'.format(train_folder))
    print('[ 20016 ] - Batch Size : {}'.format(batch_size))
    print('[ 20017 ] - Load Model Status : {}'.format(loads))
    print('[ 20018 ] - Epochs : {}'.format(epochs))
    print('[ 20019 ] - Learning Rate : {}'.format(lr))
    print('[ 20020 ] - Early Stop : {}'.format(early_stop))
    print('[ 20021 ] - Num Workers : {}'.format(num_workers))
    print('[ 20022 ] - Optimizer : {}'.format(optimizer))
    print('[ 20024 ] - Gpu Num : {}'.format(gpu_num))
    print('[ 20024a ] -Patience : {}'.format(patience))

    if gpu_num != 'CPU':
        os.environ["CUDA_VISIBLE_DEVICES"] = gpu_num # 使用编号为1, 2号的GPU
        config = tf.ConfigProto()
        config.gpu_options.allow_growth = True # 按需分配

keras.backend.tensorflow_backend.set_session(tf.Session(config=config))

    early_stop = True if early_stop.lower() == 'true' else False

    # 选择训练功能
    print('[ 20025 ] - Generate Classify Train File And Val File...')
    train_file, val_file =
self.data_processor.generate_classify_train_data(train_folder,
conf.DATA_SPLIT_RATE)
    num_class = len(conf.ANGLE_CLASSIFY_LIST)

    if loads != 'False':
        model = load_model(loads)
    else:
        model = RaKnetModel().get_classify_model(num_class)

    # 获取训练集
    print('[ 20028 ] - Generate Train Data And Split Data...')
    train_x, train_angle_y, train_throttle_y,\
    test_x, test_angle_y, test_throttle_y =
self.data_processor.trans_file_to_train_data(train_file,

val_file,

train_folder)
    current_date = datetime.datetime.now()
    current_date = 'st_{}_{}_{}'.format(current_date.month,
current_date.day, current_date.hour, current_date.minute)
    # 拼接训练文件

```



```

save_model_path = '{}_{}.mk'.format(save_model_path, current_date)
print('[ 20029 ] - Save Model Path : {}'.format(save_model_path))

# 开始训练
print('[ 20030 ] - Start Train...')
self.train(model=model,
           save_model_path=save_model_path,
           train_x=train_x,
           train_angle_y=train_angle_y,
           train_throttle_y=train_throttle_y,
           test_x=test_x,
           test_angle_y=test_angle_y,
           test_throttle_y=test_throttle_y,
           batch_size=batch_size,
           epochs=epochs,
           lr=lr,
           early_stop=early_stop,
           num_workers=num_workers,
           optimizer=optimizer,
           patience=patience)

if __name__ == '__main__':

    train_obj = RaknetTrainProcessor()

    start_time = time.time()
    parser = argparse.ArgumentParser(description='训练脚本')
    parser.add_argument('--model_name', type=str, default='monkey_car',
                        help='model name, default is monkey_car.mk')
    parser.add_argument('--train_folder', type=str, default='IADData',
                        help='Train Folder Path, default is IADData')
    parser.add_argument('--batch_size', type=int, default=conf.BATCH_SIZE,
                        help='batch size, default is 16')
    parser.add_argument('--load_model', type=str, default='False', help='load
training model, default is False')
    parser.add_argument('--epochs', type=int, default=conf.DEFAULT_EPOCHS,
                        help='epochs迭代次数')
    parser.add_argument('--lr', type=float, default=conf.INIT_LR, help='learning
rate, default if 0.0261')
    parser.add_argument('--early_stop', type=str, default='True',
                        help='early stop, default is True, you can choose
False')
    parser.add_argument('--gpu_num', type=str, default='CPU', help='GPU num,
default is CPU, first gpu number is 0')
    parser.add_argument('--num_workers', type=int, default=1, help='Num workers,
default is 1')
    parser.add_argument('--aug', type=str, default='False',
                        help='training data argument, default if False, you can
choose True')
    parser.add_argument('--optimizer', type=str, default='adam',
                        help='optimizer, default if adam, the training is
include adam, nadam, sgd')

    args = parser.parse_args()

    train_obj.go(save_model_path=args.model_name,
                 train_folder=args.train_folder,

```

```

        batch_size=args.batch_size,
        loads=args.load_model,
        epochs=args.epochs,
        lr=args.lr,
        early_stop=args.early_stop,
        num_workers=args.num_workers,
        optimizer=args.optimizer,
        gpu_num=args.gpu_num,
        patience=10)

    print('Train Speed Time:{}'.format(time.time() - start_time))

```

步骤3、编写预测代码 predict.py

```

# coding=utf-8
import cv2
import argparse
from keras.models import load_model
from Modules.model_predict import MKModelPredict

# Input Argument
parser = argparse.ArgumentParser(description='模型推理')
parser.add_argument('--model_path', type=str, default='monkey_car.mk',
                    help='model name, default is monkey_car.mk')
parser.add_argument('--img_path', type=str, default='Null', help='Predict Image,
                    Default Is Null')

args = parser.parse_args()

# Load Model
pred = MKModelPredict()
model = load_model(args.model_path)
image = cv2.imread(args.img_path)

# Model Predict
label_name, pred_conf, result_throttle = pred.predict(img=image,
                    model_pred=model)

print('Label Name: {} Conf: {} Throttle: {}'.format(label_name, pred_conf,
                    result_throttle))

'''
python predict.py --model_path xxx --img_path xxx
'''

```

训练过程：最终生成 my_weight.h5 模型文件。

```

---tub_names ./data
--train: 799, validation: 200
--steps_per_epoch 6
Epoch 1/100
D:\Anaconda3\envs\automatic_driving\lib\site-packages\pandas\core\frame.py:1490: FutureWarning: Using short name for '
FutureWarning,
2021-12-02 17:37:30.522987: I T:\src\github\tensorflow\tensorflow\core\platform\cpu_feature_guard.cc:140] Your CPU sup
1/6 [====>.....] - ETA: 28s - loss: 2.1757 - angle_out_loss: 2.4134 - throttle_out_loss: 0.3649
2/6 [=====>.....] - ETA: 20s - loss: 2.8778 - angle_out_loss: 3.1932 - throttle_out_loss: 0.3895
3/6 [=====>.....] - ETA: 13s - loss: 2.8529 - angle_out_loss: 3.1658 - throttle_out_loss: 0.3749
4/6 [=====>.....] - ETA: 9s - loss: 2.7289 - angle_out_loss: 3.0281 - throttle_out_loss: 0.3656

```

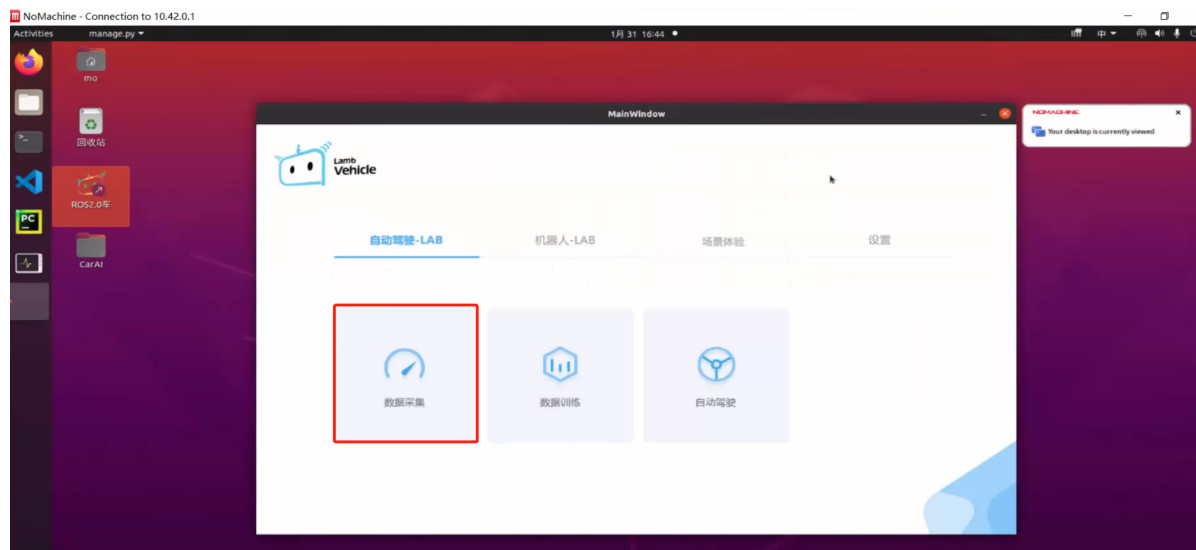
六、自动驾驶车辆实现

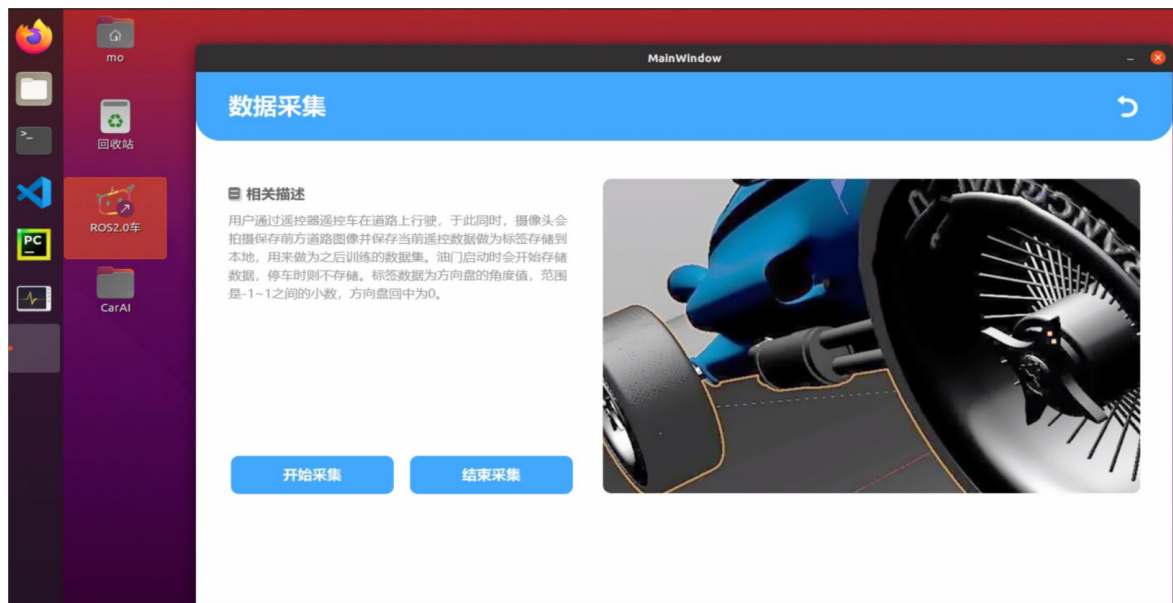
避障模型训练：通过手柄操控汽车，在放有锥桶障碍物的跑道上，行驶过程中避开锥桶，边行驶边通过摄像头采集道路数据，通过模型的训练、推理和部署，训练好的避障模型应用在自动驾驶汽车上，让汽车能够在真实环境中自动行驶。接下来就介绍如何操作自动驾驶汽车，完成整个实现过程。

1、数据采集

第一步：开始采集

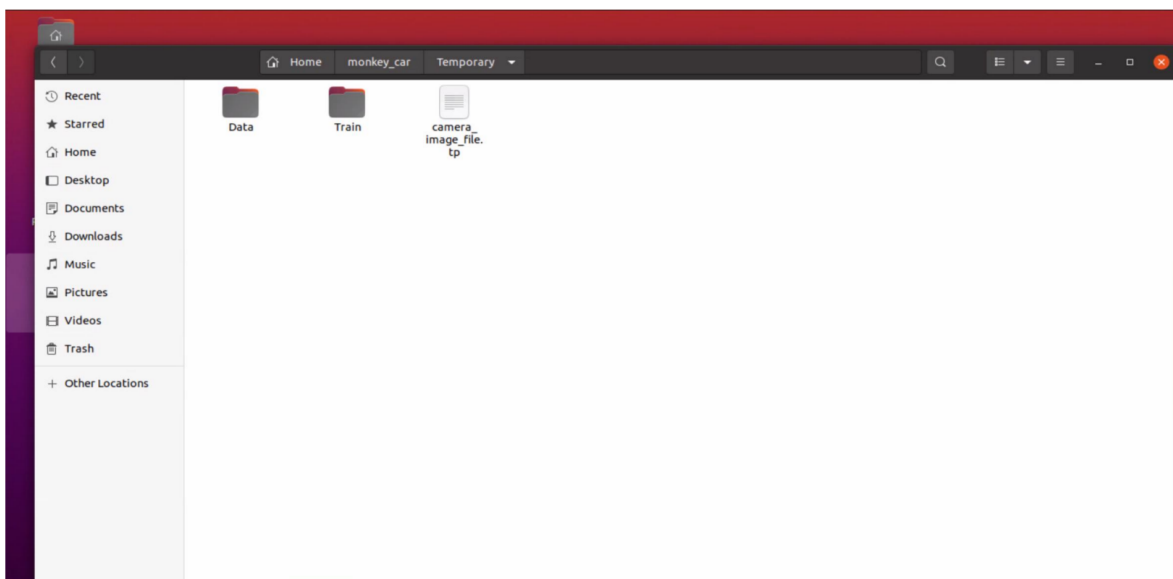
- 在LR图形化界面点击“自动驾驶-LAB”按钮，进入自动驾驶界面。点击“数据采集”按钮，启动数据采集模式。





在开始采集之前，需要确保手柄接收器已插好。若接收器未插好，点击开始采集后，车辆不会运动。点击“开始采集”按钮，用户通过随箱附带的手柄可遥控车在赛道上行驶。油门启动时会开始存储数据，停车时则不存储。

标签数据为方向盘的角度值，范围是-1~1之间的小数，方向盘回中为0。采集的数据默认存到了项目的Data文件夹里，绝对路径也就是 /Home/monkey_car/Temporary/Data



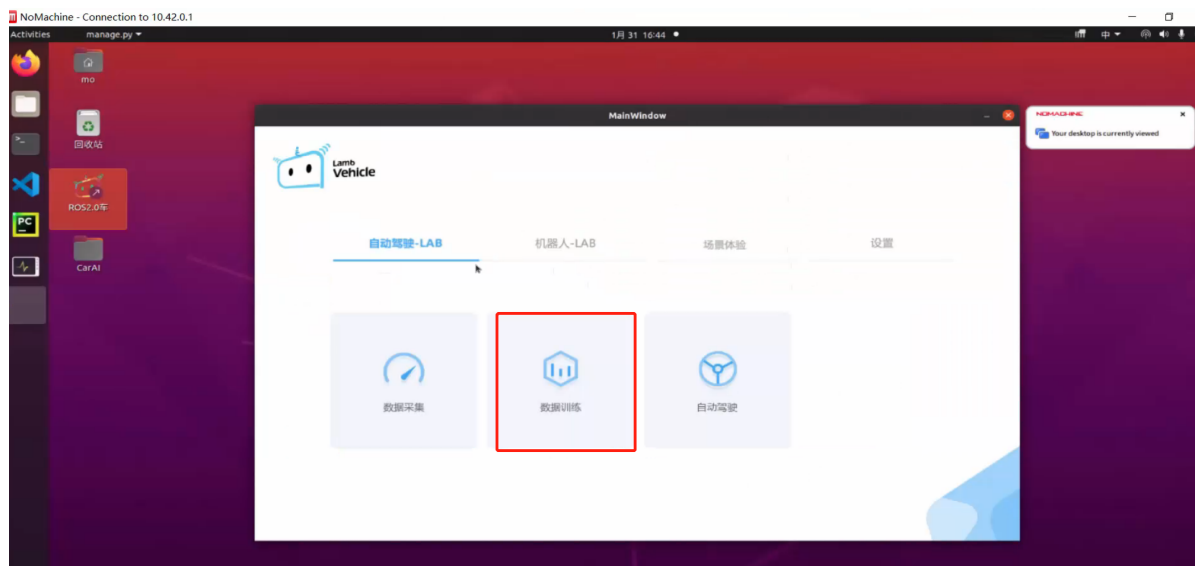
进入此文件夹会看到你采集的图片与标签，其记录的是对应前方道路图片应该的转向角度信息，这样的数据集之后会送入一个神经网络中训练生成最终的模型。故遥控驾驶小车的自动驾驶产品说明书 26 驾驶员驾驶的好坏会决定模型效果的好坏。

第二步：结束采集

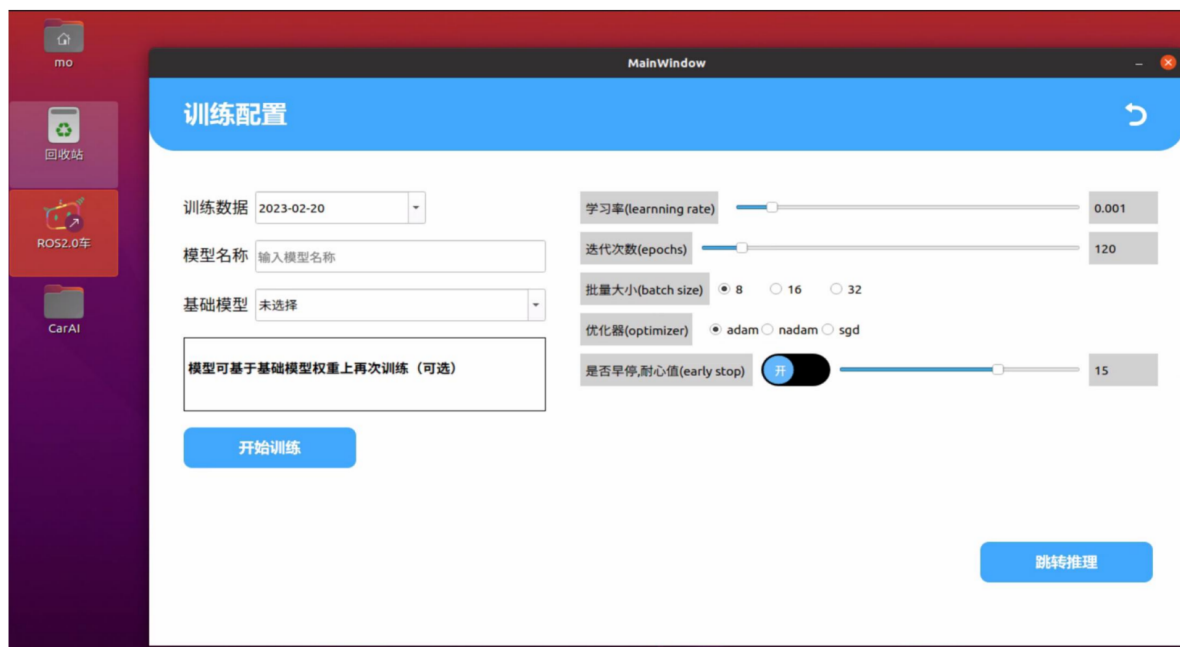
采集完成后，点击“结束采集”按钮，系统自动会在 /Home/monkey_car/Temporary/Data 文件夹下生成一个当天日期的文件夹。该系统默认不会删除以前产生的数据集，如需删除，可手动删除指定日期的数据集。当天采集的所有数据默认保存在当天日期的文件夹中。

2、模型训练

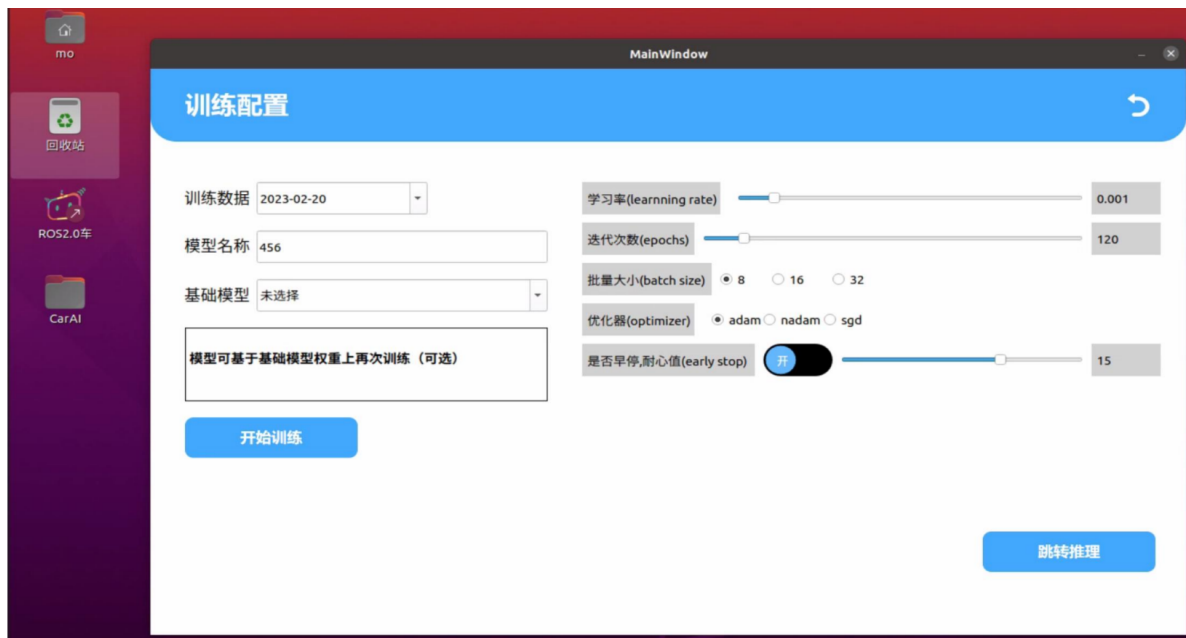
- 在图形化界面点击“数据训练”按钮进入模型训练界面。



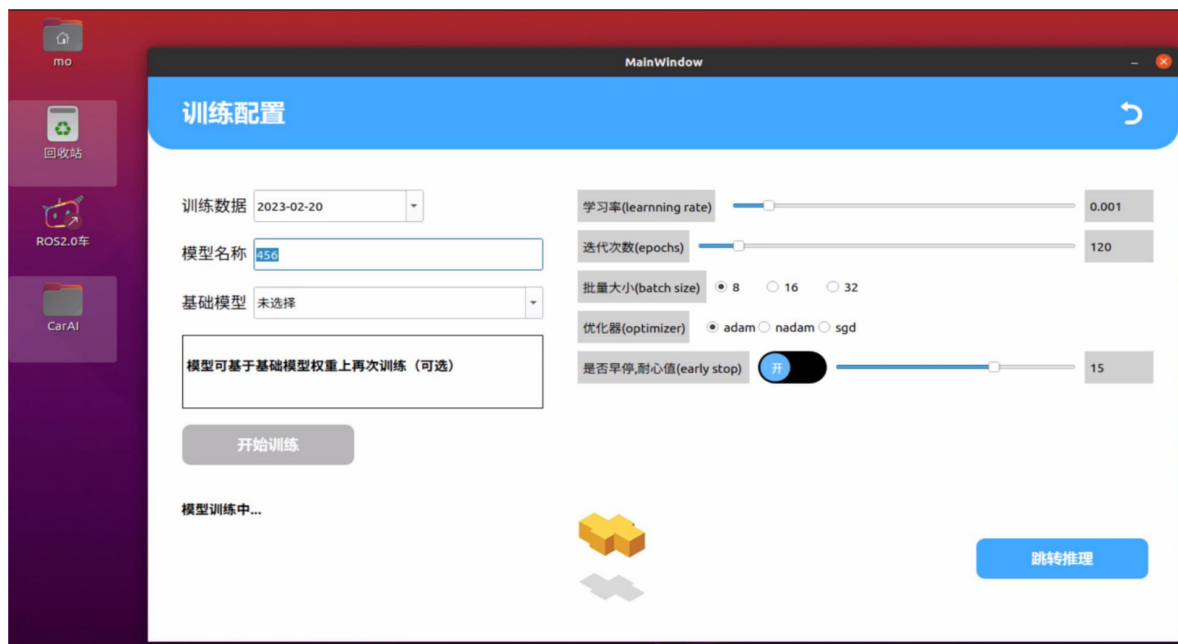
- 点击“数据训练”按钮，弹出一个对话框，进行训练配置如下图：



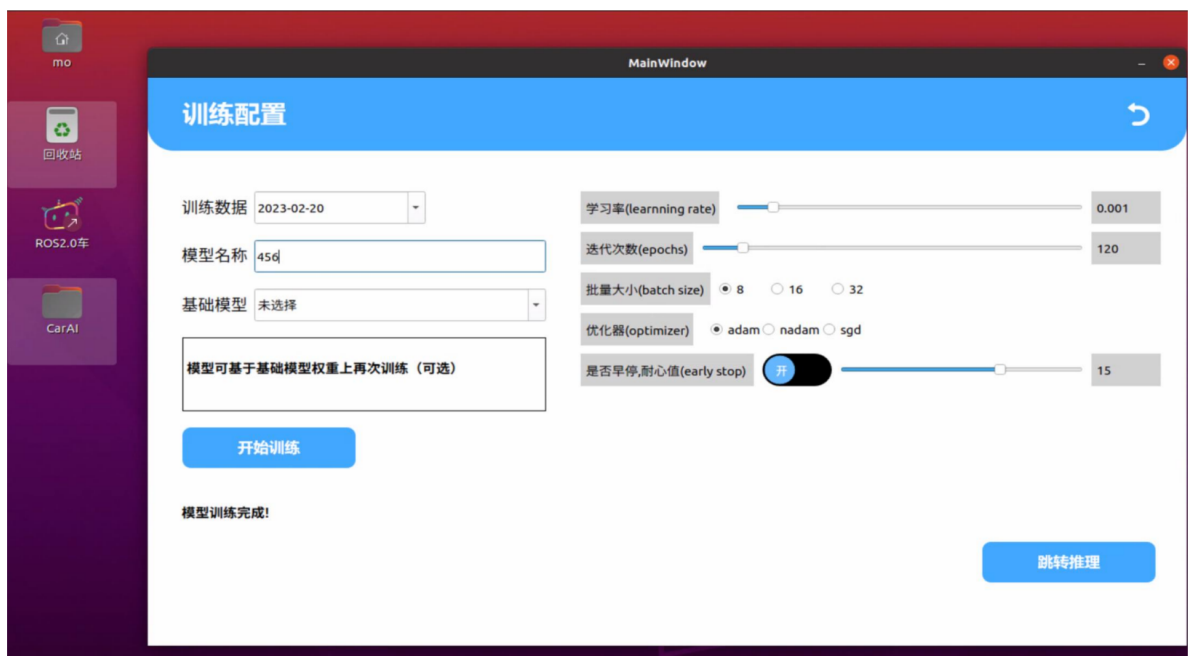
- 自定义模型名称，若进行无基础模型的训练，则直接点击“开始训练”按钮开始训练，若要进行有基础模型的训练，则在右边下拉列表框选择对应的模型后，点击开始训练后，此时开始训练按钮变为灰色不可用状态，如下图所示：



- 选择数据集：选择一个数据集进行训练。
- 模型名称：为这个数据集训练出来的模型起一个名字。
- 基础模型：如果之前有训练过的模型，想在此基础上再学习新的数据集，就选择这个模型，训练生成出来的模型会保存在/Train 文件夹里，绝对路径也就是 /Home/monkey_car/Temporary/Train。



通过采集过程的数据做为数据集，在一个回归的神经网络中进行训练，最终生成一个模型。



界面中的参数可以使用默认值，也可以进行手动调节。可手动调节项包括：学习率(lr)、训练轮次(epochs)、批量大小 (batch size)、优化器 (optimizer)、早停 (early stop)。

4、连接wifi

启动自动驾驶模式前，请将自动驾驶模型车连接到 WiFi 网络中。

连接WIFI网络步骤:

首次开机完成后，TFT 小屏幕会显示热点模式下的 IP 地址

WIFI 名称: rak_[mac 地址]

WIFI 密码: 12345678

IP 地址: 10.42.0.1 (默认)



初次连接时，网络连接中会显示需要输入 PIN 码，选择“改用安全密钥进行连接”，如下图：



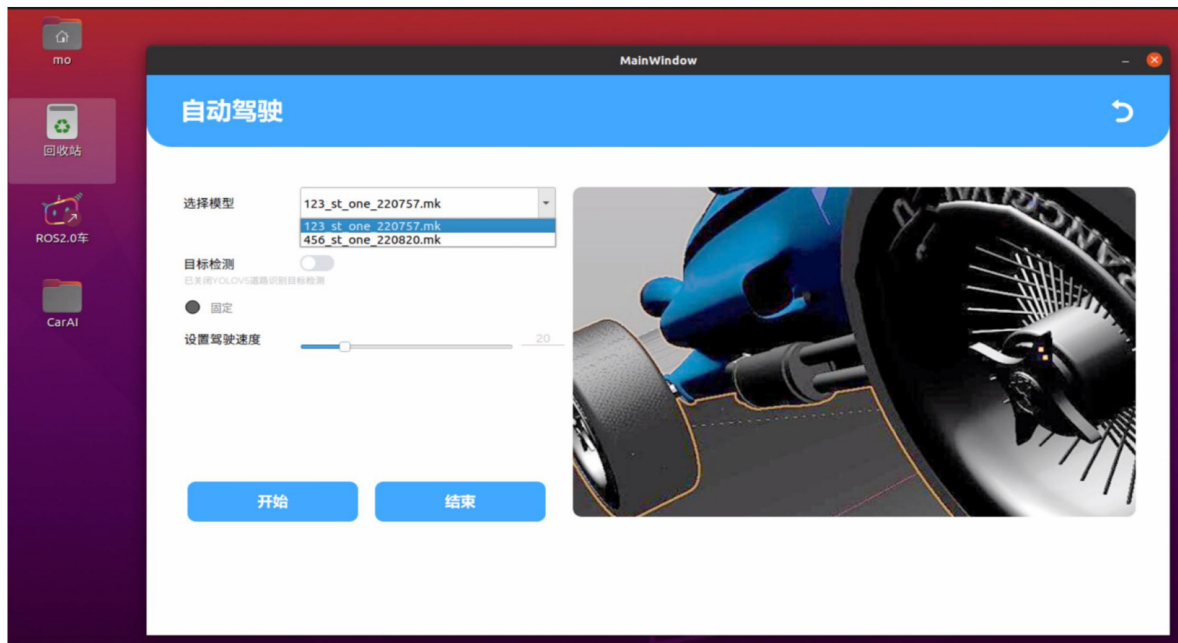
在“输入网络安全密钥”框中输入：12345678，并点击下一步按钮，如下图。



初始状态下，小车处于热点模式，可以使用远程连接但此时无法连接 Internet。

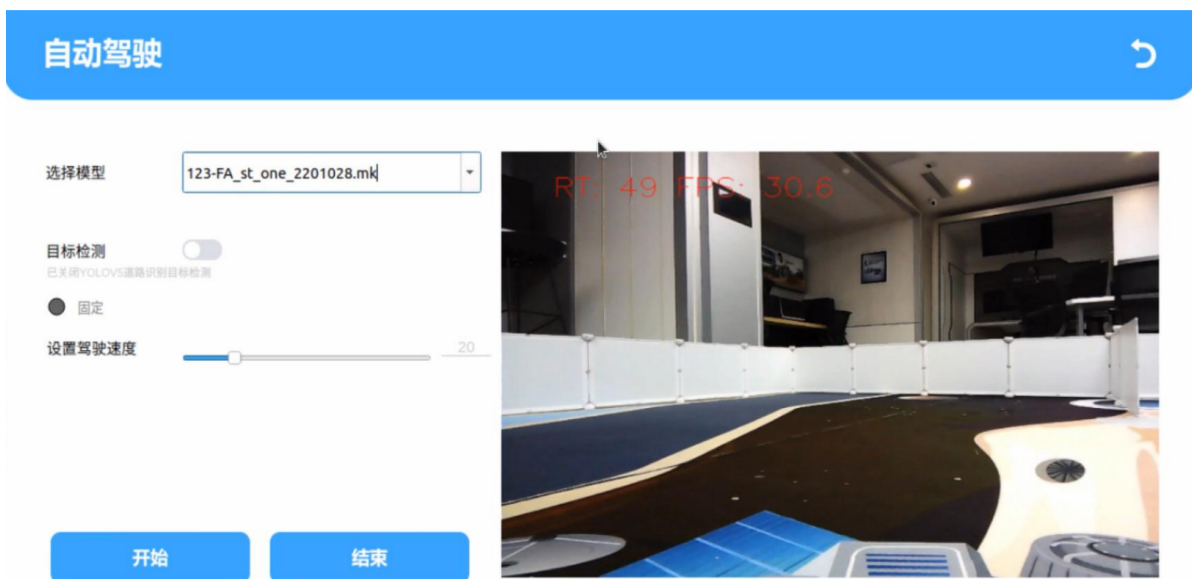
5、启动自动驾驶

- 在 LR 驾驶界面上方的模型选择框中，选择用于模型推理的自动驾驶模型，点击开始按钮



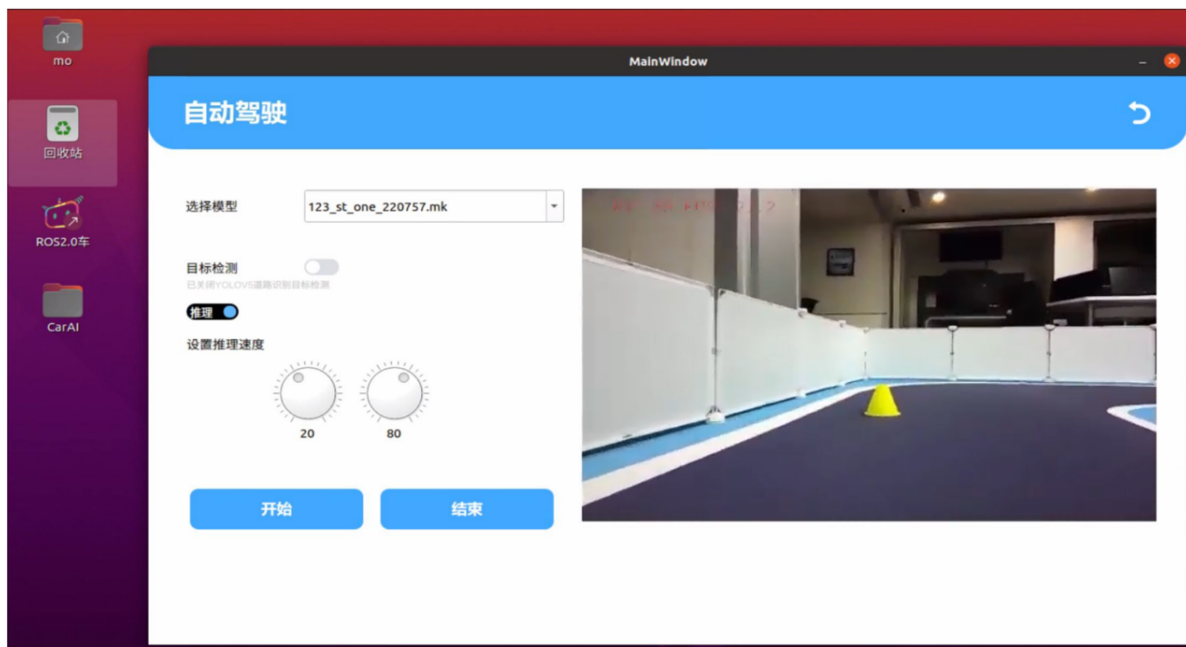
- 选择模型：选择要使用的模型
- 目标检测：在此车中预置了一个 yolov5 目标检测模型，可以识别如行人标识，停止标识，红绿灯信号等目标，如需要则开启目标检测模式。
- 运行模型：推理模型并实时输出结果，结果是一个方向盘的角度值或速度值，这个值在此车中会转换成运动方向，并向前行驶。有两种运行模式：

1) 固定速度：



软件界面中展现小车摄像头当前的视角

2) 模型推理速度



软件界面中展现小车摄像头当前的视角

- 选择模式后，可点击开始按钮，进行自动驾驶。
- 点击结束按钮后，小车停止运行，界面显示为小车当前视角。

七、实验总结

通过卷积神经网络CNN，实现车辆行驶路线规划并自动实现避障，卷积的核心是数据降维，提取图像核心特征、完整的CNN网络结构：输入层、卷积层、池化层、全连接层、输出层。各层的数量可以随意的定制，我们实验中就构建了9层网络实现模型构建。